ChangeGuard — Platform Architecture & SRE Field Guide

AI-Powered Pre-Deployment Change-Risk Intelligence & Autonomous Incident Remediation

SYSTEMS ENGINEERING & RELIABILITY PLATFORM | MODEL CONTEXT PROTOCOL (MCP) · VECTOR RAG · GITOPS | V2.4 PRODUCTION SPECIFICATION

1. Executive Summary & Core Philosophy

In modern high-scale distributed systems, **over 80% of critical production outages are triggered by human code deployments and configuration changes**, rather than random hardware or network failures. When software degrades in real time, traditional approaches fall short: manual human code reviews fail to catch non-obvious latency regressions, and on-call engineers are paged at 3:00 AM to manually dig through disjointed logs, metrics, and Git commits to identify what broke.

ChangeGuard is a production-grade DevOps and AI reliability platform designed to solve both sides of this problem using a connected-vehicle telemetry pipeline as its mission-critical target system:

1. **Shift-Left Pre-Deployment Safety Gate (Change-Risk Engine):** Analyzes incoming Pull Requests and Git diffs prior to merging. It calculates risk scores (0–100), detects anti-patterns (unbounded queries, thread-blocking sleeps, memory leaks, hardcoded credentials), maps downstream blast radius, and recommends canary rollout strategies.
2. **Shift-Right Autonomous Incident Remediation (AI SRE Agent):** When live telemetry degrades or Prometheus alerts fire, an autonomous AI SRE agent leverages the **Model Context Protocol (MCP)** and a **Vectorized RAG Knowledge Base** to inspect metrics, grep container logs, correlate recent Git blame, determine root cause, and execute a verified GitOps rollback.

2. Mental Models & Layman Analogies

To communicate the value and mechanics of ChangeGuard intuitively, four foundational analogies describe each subsystem:

> 🛬 **Analogy 1: The Airport Baggage X-Ray Scanner (The Change-Risk Engine)**
> Before passengers board an airplane, their luggage passes through an automated X-ray security scanner. The scanner does not care who owns the bag; it systematically flags dangerous items (explosives, weapons, liquids).
> **In ChangeGuard:** Before code reaches production, the Risk Engine scans every line of the Git diff. It flags "dangerous contraband" in code—such as an unindexed `SELECT *` query, a thread-freezing `time.sleep()`, or an unmanaged memory cache—stopping faulty code at the gate before it can crash production servers.

🩺 Analogy 2: The Emergency Room Specialist (The Autonomous AI SRE Agent)

When a patient arrives in the ER, doctors do not guess blindly. They follow an established diagnostic protocol: check vitals (heart rate/blood pressure), order focused bloodwork, review medical history, identify the exact ailment, and administer the treatment.

In ChangeGuard: When vehicle telemetry degrades, the AI SRE Agent checks vitals (Prometheus metrics), inspects symptoms (container logs), inspects what changed recently (Git commits), and administers treatment (GitOps rollback to stable release).

🔌 Analogy 3: Universal USB-C Port for AI (Model Context Protocol - MCP)

Instead of every phone having a custom, fragile charging cable, USB-C provides a single standardized plug that transfers power and data safely across all devices.

In ChangeGuard: Instead of giving an AI model raw terminal bash access (which can accidentally execute destructive commands like `rm -rf`), **MCP** provides a clean, typed set of tools. The agent safely calls `query_prometheus()`, `git_log()`, or `trigger_rollback()` through standard JSON contracts.

📖 Analogy 4: The Black Box Flight Recorder & Incident Archive (Vector RAG)

Commercial aviation is exceptionally safe because every time an aircraft encounters a mechanical issue, an official National Transportation Safety Board (NTSB) report is published and archived so pilots never make that mistake again.

In ChangeGuard: Top tech companies archive post-mortem reports. ChangeGuard's **Vectorized RAG Engine** indexes past production incidents. When a new latency spike occurs, the AI searches past post-mortems and applies proven historical remediations rather than hallucinating.

3. What is an SRE in the Real World?

The term **Site Reliability Engineer (SRE)** was coined at Google by Ben Treynor Sloss with the guiding definition: "*SRE is what happens when you ask a software engineer to design an operations team.*"

Role	Primary Question Asked	Day-to-Day Responsibilities	Incident Response Approach
Software Engineer (SWE)	"*How do I build this new feature?*"	Writes application logic, implements UX, designs APIs, delivers product roadmap milestones.	Fixes application bugs during business hours after tickets are filed.
Traditional SysAdmin	"*How do I keep this server running?*"	Manages physical hardware, OS updates, manual server configuration, periodic reboots.	Manual triage, SSH into machines, manual restarts, edits live config files.
Site Reliability Engineer (SRE)	"*How do I write software that automates system reliability?*"	Treats operations as a software problem. Designs SLIs/SLOs, automates deployment gates, manages chaos tests, eliminates operational toil.	Automated alerting, telemetry correlation, git blame attribution, automated rollbacks, and blameless post-mortems.

Core SRE Terminology

- **SLI (Service Level Indicator):** A quantifiable metric measured in real time (e.g., "*p95 telemetry ingestion latency is currently 42ms*").
- **SLO (Service Level Objective):** The target threshold agreed upon with the business (e.g., "*99.9% of telemetry requests must respond in under 100ms*").
- **SLA (Service Level Agreement):** The contractual commitment to customers with financial penalties if violated.
- **Error Budget:** The allowable room for failure (100% - SLO). If the error budget is exhausted, deployments are automatically frozen.
- **MTTR (Mean Time to Recovery):** The average time required to detect, diagnose, and resolve an outage. ChangeGuard reduces MTTR from 45 minutes to under 60 seconds.

4. Pre-Deployment Gate: The Change-Risk Engine

The Change-Risk Engine (`:8001`) operates as an automated pre-deployment safety gate. When a developer submits a Pull Request, the engine analyzes the raw Git unified diff and executes three evaluation phases:

Phase 1: File Categorization & Criticality Weighting

The parser categorizes files modified in the diff using regular expression pattern classifiers:

- **Database & Schema:** `.sql` , `migrations/` , `schema.` , `models.py` → **+35 Base Risk** (affects persistent storage).
- **Infrastructure & Deployment:** `Dockerfile` , `docker-compose` , `.tf` , `.yaml` , `k8s/` → **+25 Base Risk** (affects container orchestration).
- **Security & Authentication:** `auth` , `jwt` , `permission` , `secret` , `token` → **+25 Base Risk** (touches access control).

- **Core Ingestion Pipeline:** `telemetry`, `ingest`, `stream`, `kafka`, `mqtt` → Evaluated for throughput bottlenecks.
- **Third-Party Dependencies:** `requirements.txt`, `package.json`, `go.mod` → **+15 Base Risk** (supply-chain updates).
- **Documentation:** `.md`, `.rst`, `docs/` → **0 Base Risk** (safe for immediate auto-approval).

Phase 2: Dangerous Code Pattern Detection

Detected Pattern	Penalty	Production Failure Mode Prevented
`DROP TABLE` | `ALTER TABLE`	**+40 Points**	Destructive DDL migrations locking tables during peak ingestion hours, causing data loss.
`SELECT * FROM` without `LIMIT`	**+30 Points**	Unbounded queries triggering full table scans, exhausting database connection pools and memory.
`time.sleep()` | `sleep()`	**+25 Points**	Synchronous blocking calls inside asynchronous event loops, causing worker thread starvation.
`password` | `secret` | `api_key = "..."`	**+35 Points**	Hardcoded secrets committed into version control, risking credential exfiltration.
`verify=False` | `REJECT_UNAUTHORIZED=0`	**+35 Points**	Disabled TLS/SSL certificate verification, exposing vehicle data to Man-In-The-Middle attacks.
`timeout=None` | missing timeout	**+25 Points**	External network calls with infinite timeout, causing cascading thread deadlocks.
`while True:` | unbounded loop	**+20 Points**	CPU lockup and runaway loop execution starving sibling requests.
`memory_buffer.append()` (unmanaged)	**+25 Points**	Unbounded in-memory accumulation triggering OS Out-Of-Memory (OOM) killer terminations.

5. Risk Scoring & Blast Radius Topology

The 0–100 Risk Score & Governance Tiers

The cumulative risk score dictates the governance gate enforced on the Pull Request:

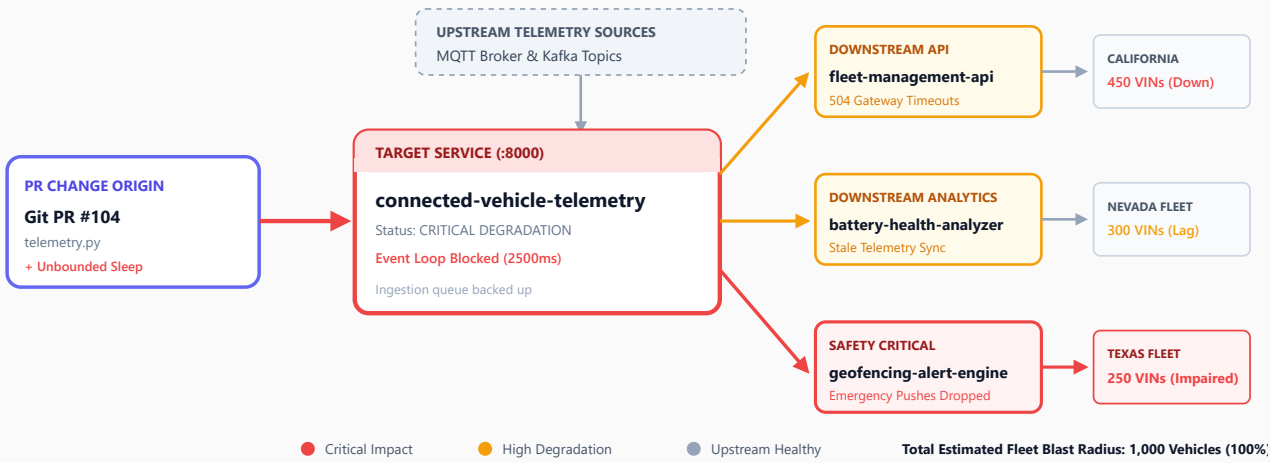
Score Range	Risk Tier	Approval Status	Mandated Canary & Deployment Strategy
0 – 24	LOW	`AUTO_APPROVED`	Standard rolling deployment with automated endpoint health probes.
25 – 49	MEDIUM	`REQUIRES_PEER_REVIEW`	Blue/Green deployment with instant automated traffic cutback gate.
50 – 74	HIGH	`REQUIRES_HUMAN_SIGN_OFF`	Standard Canary (5% → 50% → 100%) with 15-minute soak and SRE lead sign-off.
75 – 100	CRITICAL	`BLOCKED`	Deployment blocked at CI gate. Multi-stage Canary (1% → 5% → 25%) with 1-hour bake time.

Blast Radius & Dependency Topology

When a microservice changes, the Risk Engine consults the internal service graph to compute the **Blast Radius**:

- **Originating Service:** `connected-vehicle-telemetry`
- **Direct Downstream Dependents:** `fleet-management-api`, `battery-health-analyzer`, `geofencing-alert-engine`.
- **Upstream Ingestion Dependencies:** `mqtt-broker`, `kafka-telemetry-topic`.
- **Impact Projection:** If telemetry fails, 100% of the California, Nevada, and Texas vehicle fleets experience degraded tracking, and geofencing safety alarms are impaired.

FIG 5.1 — BLAST RADIUS SERVICE DEPENDENCY TOPOLOGY & FLEET IMPACT GRAPH



6. The 7 Production Scenarios (Code & Analysis)

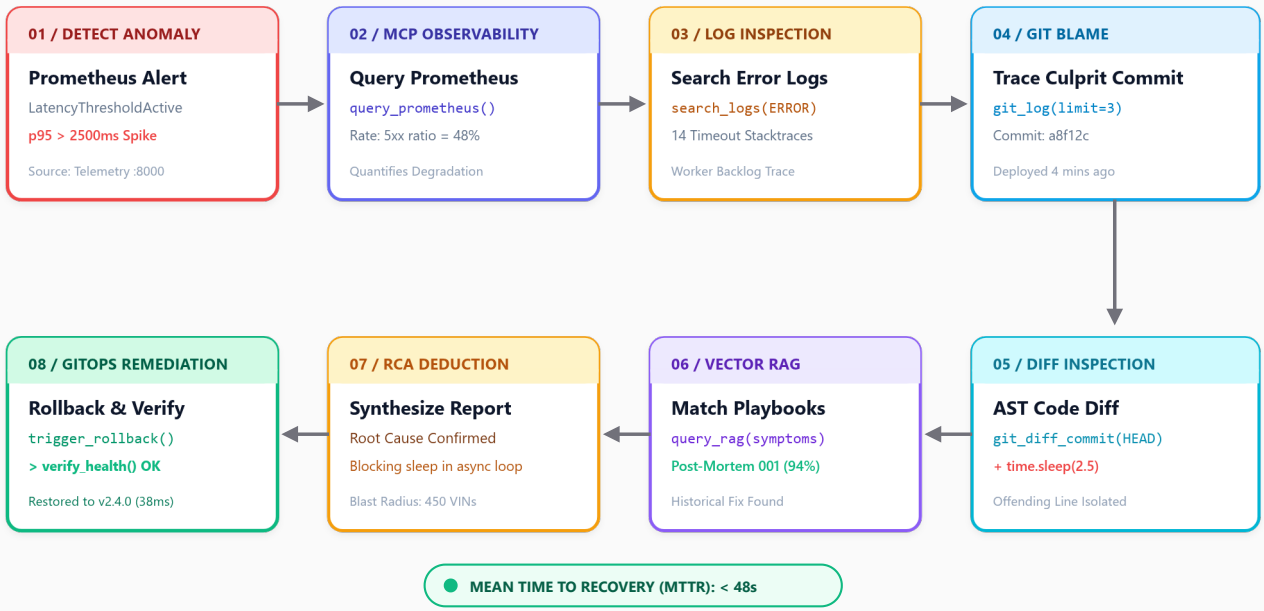
ChangeGuard includes 7 distinct production scenarios covering standard engineering risks:

Scenario Name	Risk Tier	Core Anti-Pattern / Code Flaw	Blast Radius & Impact
1. Unbounded Query & Sleep	HIGH (65/100)	`SELECT *` without index/limit + `time.sleep(2.5)` in ingestion path.	Worker backlog, request timeouts across vehicle fleets.
2. Memory Buffer Leak	CRITICAL (75/100)	`global_coordinate_cache.append(bytearray(50MB))` without eviction.	Rapid RAM consumption leading to Linux OOM killer panic.
3. API Key & No SSL	CRITICAL (95/100)	Hardcoded `sk_live_...` API key and `verify=False` on external sync.	Credential leak risk and Man-in-the-Middle vulnerability.
4. Destructive Table Drop	CRITICAL (85/100)	`DROP TABLE vehicle_historical_events;`	Irreversible data loss; database lock cascading across all microservices.
5. Missing Timeout	HIGH (55/100)	`httpx.Client(timeout=None)` in a `while True` loop.	Network socket exhaustion and cascading thread starvation.
6. Optimized Feature	LOW (10/100)	Strict `LIMIT 10`, indexed query, parameterized SQL statement.	Safe, high-efficiency execution. Auto-approved.
7. Safe Documentation	LOW (10/100)	Edits to `README.md` specifications only.	Zero runtime risk. Auto-approved.

7. Shift-Right Remediation: The Autonomous AI SRE Agent

When degradation bypasses pre-deployment gates or chaos faults are injected, the Autonomous AI SRE Agent (`:8003`) initiates a structured 8-step investigative diagnosis. Below is the visual state graph tracing the autonomous workflow from alert detection to GitOps rollback:

FIG 7.1 — AUTONOMOUS AI SRE INVESTIGATION & REMEDIATION STATE GRAPH



8. Model Context Protocol (MCP) Tool Registry

ChangeGuard's MCP Server (`:8002`) exposes 10 standardized, strictly typed operational tools:

MCP Tool Name	Input Arguments	Operational Output & Purpose
`get_active_alerts`	None	Returns all currently firing Prometheus alerts and alert metadata.
`query_prometheus`	`query` (PromQL string)	Executes PromQL time-series metric queries against Prometheus.
`search_logs`	`level`, `keyword`	Searches recent application container logs for stack traces and errors.
`git_log`	`limit` (int)	Retrieves recent commits, authors, hashes, and commit messages.
`git_diff_commit`	`commit_hash` (string)	Extracts the exact unified diff introduced by a specific commit.
`git_blame`	`file_path`, `line_no`	Identifies the developer and commit responsible for specific lines.
`list_containers`	None	Lists all running Docker containers, status, and port bindings.
`inspect_container`	`container_name`	Inspects container CPU, memory usage, environment variables, and uptime.
`trigger_rollback`	`service_name`, `target_version`	Executes an automated GitOps rollback to a verified stable revision.
`verify_health`	`service_name`	Probes health endpoints post-rollback to verify full recovery.

9. Vectorized RAG Incident Playbooks

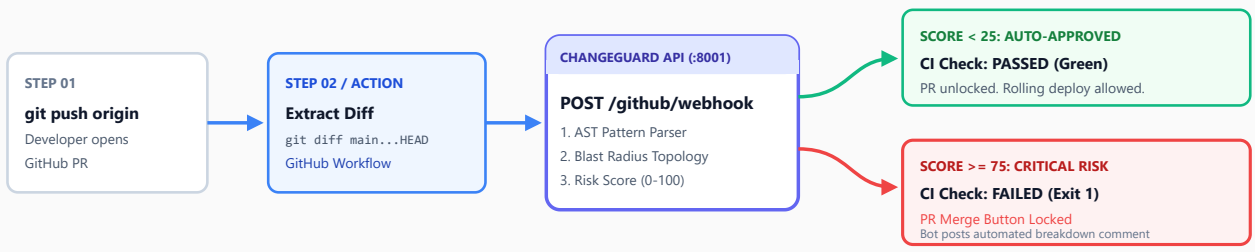
ChangeGuard's RAG Knowledge Engine (`:8004`) converts historical incident reports into TF-IDF vector embeddings with cosine similarity matching. The repository indexes 8 production post-mortems:

1. **Post-Mortem 001:** P1 Ingestion Latency Degradation in California Fleet (blocking sleep in event loop).
2. **Post-Mortem 002:** Global Memory Buffer Leak (unbounded bytearray coordinate cache).
3. **Post-Mortem 003:** PostgreSQL Connection Pool Starvation (unclosed connection handles).
4. **Post-Mortem 004:** Environment Configuration Drift (missing port mapping in deployment manifest).
5. **Post-Mortem 005:** Cascading Network Timeout (missing circuit breaker on geocoding client).
6. **Post-Mortem 006:** Kafka Telemetry Topic Backpressure (consumer group lag spike).
7. **Post-Mortem 007:** Canary Ingestion Error Cascade (unhandled null VIN telemetry packet).
8. **Post-Mortem 008:** Unindexed Range Query Performance Degradation (full table scan on timestamps).

10. GitHub CI/CD & Webhook Integration

In production environments, no human copies and pastes diffs manually. ChangeGuard integrates directly into GitHub via **GitHub Actions** and **Webhooks**. The flow graph below illustrates the automated PR lifecycle from developer commit to deployment gating:

FIG 10.1 — GITHUB CI/CD REAL-TIME PULL REQUEST RISK GATING WORKFLOW



```
# .github/workflows/changeguard-risk-gate.yml
name: ChangeGuard AI Pre-Deploy Risk Gate
on:
  pull_request:
    types: [opened, synchronize, reopened]

jobs:
  risk-assessment:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - name: Extract PR Unified Diff
        run: git diff origin/${{ github.base_ref }}...HEAD > pr.diff
      - name: Call ChangeGuard Risk Gate
        run: |
          RESPONSE=$(curl -s -X POST "https://changeguard.internal:8001/github/webhook" \
            -H "Content-Type: application/json" \
            -d "{\"pull_request\": {\"title\": \"${{ github.event.pull_request.title }}\", \"numb
er\": ${{ github.event.pull_request.number }}}, \"diff\": $(jq -s -R . < pr.diff)}")
          STATUS=$(echo "$RESPONSE" | jq -r '.risk_assessment.approval_status')
          if [ "$STATUS" = "BLOCKED" ]; then
            echo "PR Blocked by ChangeGuard! Critical risk detected." && exit 1
          fi
```

11. Deployment Architecture Summary

Service Name	Container Port	Technology	Core Responsibilities
dashboard	`:3000`	Nginx / HTML5 / CSS3	Real-time operations center, live fleet stream, chaos studio, risk analyzer.
telemetry-service	`:8000`	FastAPI / Python 3.12	High-frequency vehicle ingestion, GPS/speed streaming, chaos injection API.
risk-engine	`:8001`	FastAPI / AST Parser	Diff parsing, blast radius evaluation, risk scoring, GitHub webhook.
mcp-server	`:8002`	FastAPI / MCP Protocol	10 typed diagnostic tools exposing Prometheus, Git, Docker, and Logs.
sre-agent	`:8003`	FastAPI / Autonomous Agent	8-step investigative diagnosis loop, RCA synthesis, rollback orchestration.
rag-engine	`:8004`	FastAPI / Scikit-Learn	Vectorized incident post-mortem indexing and cosine similarity retrieval.
prometheus	`:9090`	Prometheus v2.54.1	Time-series metric scraping, alert threshold evaluation.
grafana	`:3001`	Grafana v11.1.0	Pre-provisioned vehicle telemetry and cluster health dashboards.